나의 전제(Beliefs)

나의 전제: AI를 사용할 때 받아들이고 있는 세 가지 믿음.

비판: LLM은 확률론적 앵무새(Stochastic parrot)일 뿐이다

→ 예측은 앵무새의 증거가 아니다. 우리 뇌도 예측으로 언어를 처리한다.

비판: AI를 쓰면 사고력과 추론 능력이 떨어진다

→ AI는 증폭기다. 생각을 맡기면 줄고, 생각을 넓히는 데 쓰면 는다.

비판: AI는 평균적인 결과물만 만든다

→ AI는 평균 이상의 초안을 모두에게 비슷하게 준다. 완성은 사람이 한다.

AI는 예측으로 돕고, 사람은 판단으로 완성한다.

경쟁 상대는 줄지 않는다. 진짜 경쟁 상대는 더 똑똑해진다.

예측(Prediction)

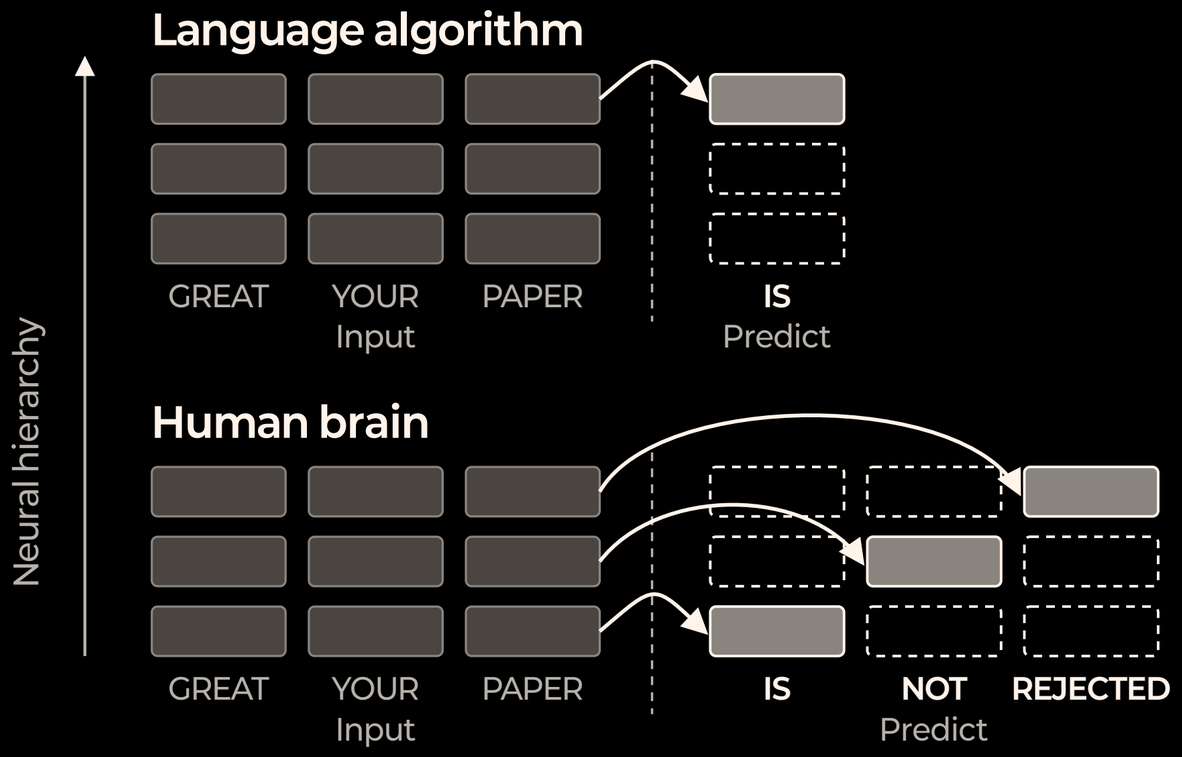
예측: 앞선 맥락으로 다음 말을 추정하는 과정. 사람의 뇌도 언어를 이렇게 처리한다.

비판: LLM은 의미 없이 형식만 잇는 확률론적 앵무새(Stochastic parrot)다.

뇌 영상 연구에서 사람의 뇌도 다음 단어를 예측하며, 언어 모델보다 더 먼 단어를 여러 층위에서 예측한다.

예측한다는 사실만으로 이해가 없다고 보면, 사람의 이해도 부정하게 된다.

다만 학습 비용, 데이터 편향, 오용 위험에 대한 원 논문의 지적은 유효하다.



언어 모델은 다음 한 단어를, 뇌는 여러 층위에서 더 먼 단어까지 예측

출처: Caucheteux, Gramfort & King(Nature Human Behaviour, 2023)

증폭(Amplification)

증폭: AI가 사고를 선택으로 만들면서, 쓰는 방식에 따라 능력 격차가 벌어지는 현상.

비판: AI를 쓰면 사고력과 추론 능력이 떨어진다

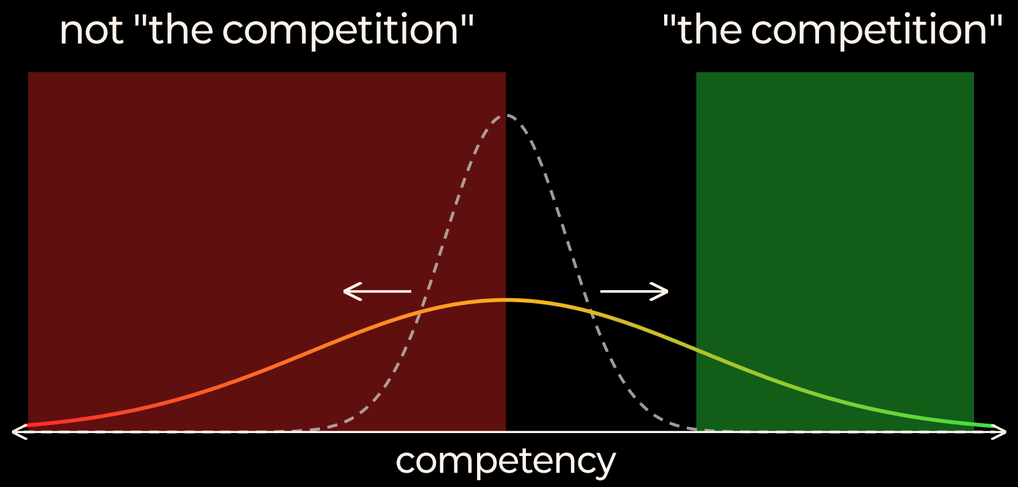
고생이 훈련 신호다. 생각을 AI에 맡기면
부하가 사라지고 능력도 줄어든다.

그러나 AI는 훈련 장비도 된다

체스 엔진 이후 사람의 실력이 오른 것처럼,
개념을 묻고 비판을 받는 데 쓰면 는다.

※ 양극화(Bifurcation)

분포가 가운데로 모이지 않고
양쪽으로 벌어지는 것



점선은 AI 이전, 색 곡선은 AI 이후의 역량 분포, 양쪽으로 벌어짐
출처: Eero Alvar, “Bodybuilding for the Mind”(YouTube, 2026)

완성(Completion)

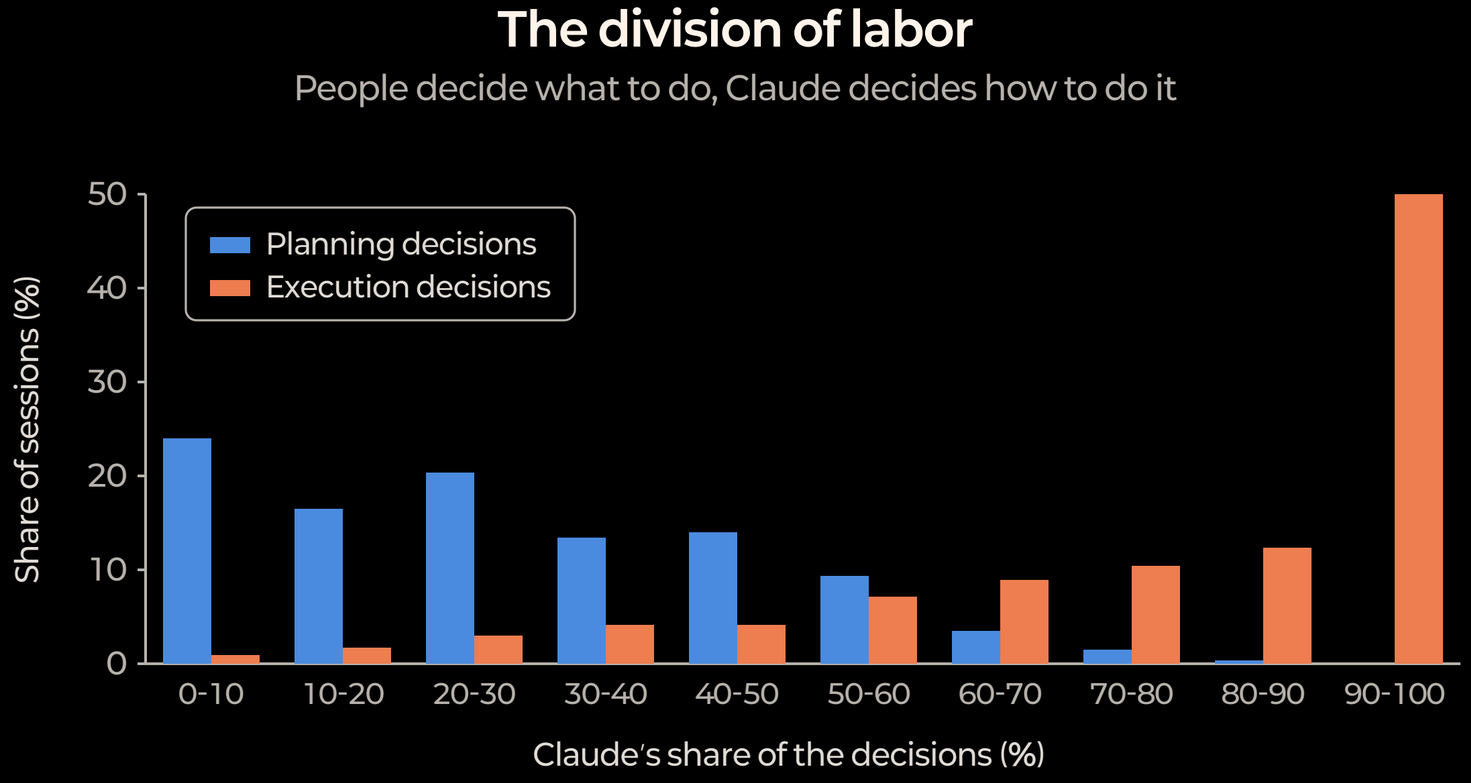
완성: 초안에서 무엇을 남기고 고칠지 판단해 결과물을 끝내는 일.

비판: AI는 평균적인 결과물만 만든다.

AI를 쓰면 개인의 창의성 평가는 오르지만, 결과물끼리는 서로 비슷해진다.

Claude Code 사용에서도 **무엇을 할지는 사람이, 어떻게 할지는 AI가 정한다.**

초안은 AI가 제공하며, 선택과 판단, 마지막 완성과 그 고생은 사람의 몫이다.



계획 결정은 대부분 사람이, 실행 결정은 대부분 Claude가 맡음
출처: Anthropic, Claude Code 사용 연구(2026)

사람 개입(Human in the loop)

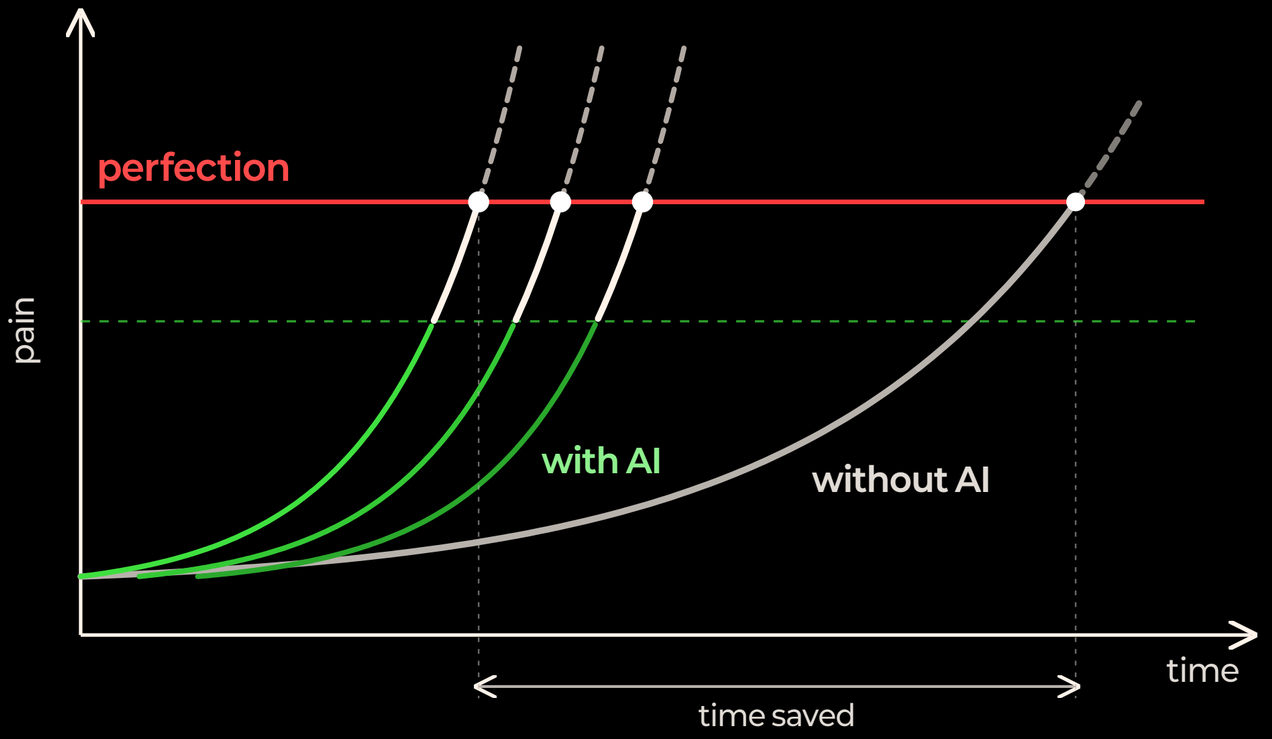
사람 개입: AI의 작업 과정에 사람이 판단과 검증으로 참여하는 것.

AI는 결과물의 대부분을 빠르게 만들지만, 마지막 구간은 여전히 사람의 판단으로 채워진다.

완성에 가까워질수록 한 단계를 올리는 데 드는 노력은 줄지 않는다. 데모는 쉽고, 완성은 어렵다.

AI가 줄여 준 시간만큼 여러 작업을 동시에 진행할 수 있으며, 이때 병목은 작성에서 검토로 옮겨간다.

※ 나인의 행진(March of nines): 신뢰도를 90%에서 99%, 99.9%로 한 단계 올릴 때마다 비슷한 노력이 드는 현상



AI는 기준 아래까지 빠르게, 남은 구간은 사람이 채움, 여러 과정 병렬 진행

메타인지 부담(Metacognitive demands)

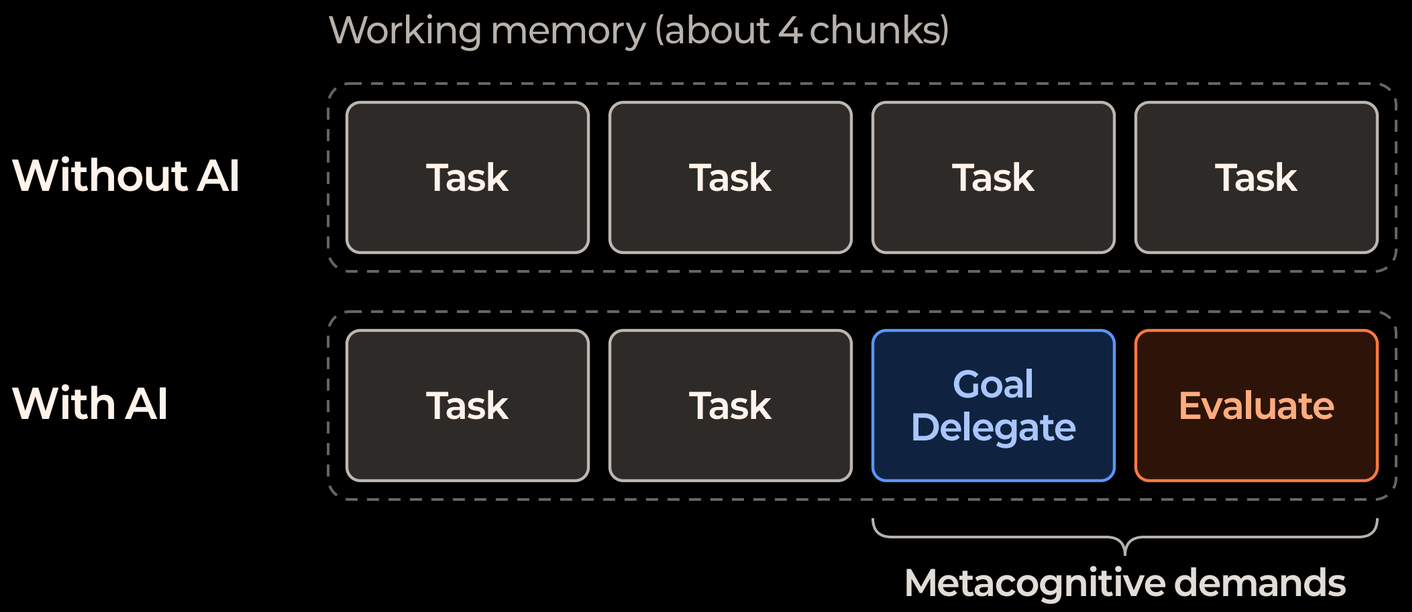
메타인지 부담: AI와 일하는 동안 무엇을 원하고 맡길지, 결과가 맞는지 계속 판단해야 하는 부담.

AI가 작업을 대신하는 만큼, 목표를 정하고 일을 나누고 결과를 평가하는 생각은 사람에게 남는다.

이 생각이 작업 기억을 함께 차지하면서, **정작 문제를 깊이 생각할 여유가 줄어든다.**

원하는 것을 정리하지 않고 먼저 물으면, **그 생각 자체를 건너뛰게 된다.**

※ 메타인지(Metacognition): 자신의 생각을 살피고 조절하는 생각



AI와 일하면 작업 기억의 일부를 목표 설정과 결과 평가가 차지

표현 방식(Representation)

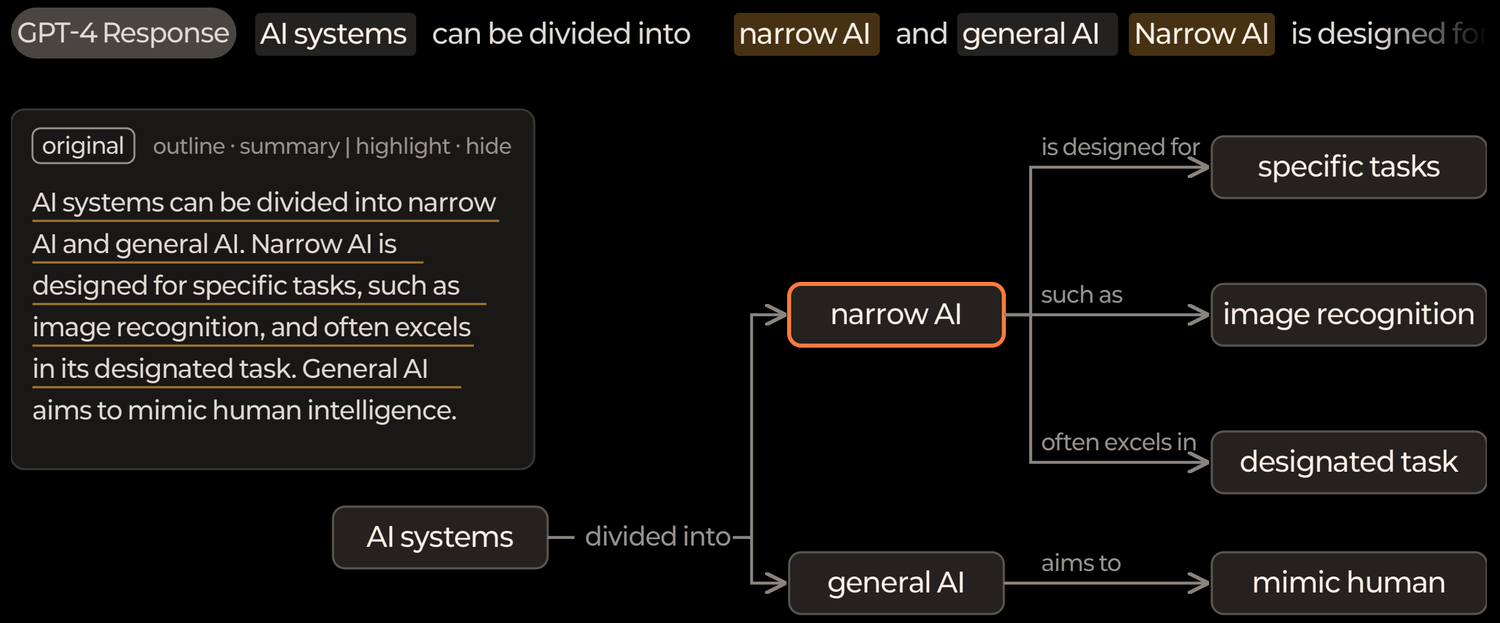
표현 방식: AI의 결과를 사람이 빠르게 이해하고 검증할 수 있는 형태로 바꾸어 보여 주는 것.

AI의 출력은 읽는 속도보다 빠르고 길며, 구조가 드러나지 않아 해석하는 부담이 크다.

같은 내용도 도식, 표, 결론 우선 요약으로 보면 **탐색이 넓어지고 오류를 더 잘 찾는다.**

결과를 받기 전에 **보여 줄 형식을 정해 요청하는 것도 사람의 일이다.**

※ 생성형 UI(Generative UI): 질문에 맞춰 AI가 화면 구성까지 만들어 보여 주는 방식



긴 답변 글을 개념과 관계의 도식으로 바꾸어 보여 줌
출처: Jiang 외, Graphologue(UIST 2023)

AI 에이전트(AI agent)

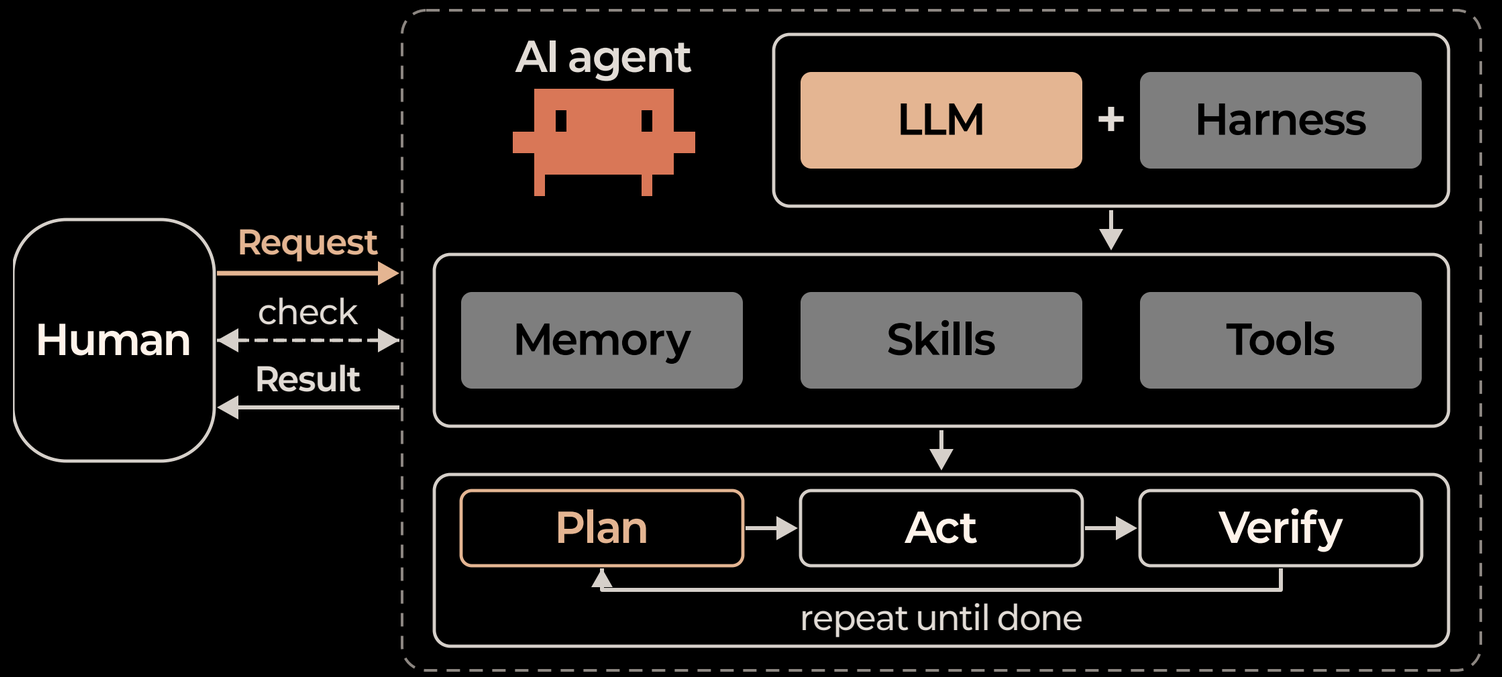
AI 에이전트: 목표를 받아 도구를 반복해 쓰며 계획·실행·검증하는 LLM 시스템.

에이전트는 LLM과 하네스로 이루어진다. LLM이 다음 행동을 판단하면, 하네스가 이를 실행하고 결과를 돌려준다.

요청을 받으면 LLM이 기억(지침 파일·작업 기록), 스킬(작업 절차), 도구(파일·셸·검색) 중 필요한 것을 골라 쓴다.

사람이 목표를 주면 에이전트는 **계획, 실행, 검증을 끝날 때까지 반복하고,** 완료되면 결과를 보고한다.

※ 하네스(Harness): 모델을 감싸 도구를 제공하고 컨텍스트를 관리하는 프로그램 (Claude Code, Codex 등)



사람의 요청에 기억·스킬·도구를 골라 쓰며 계획·실행·검증 반복, 결과 보고

도메인 특화 에이전트(Domain-specific agent)

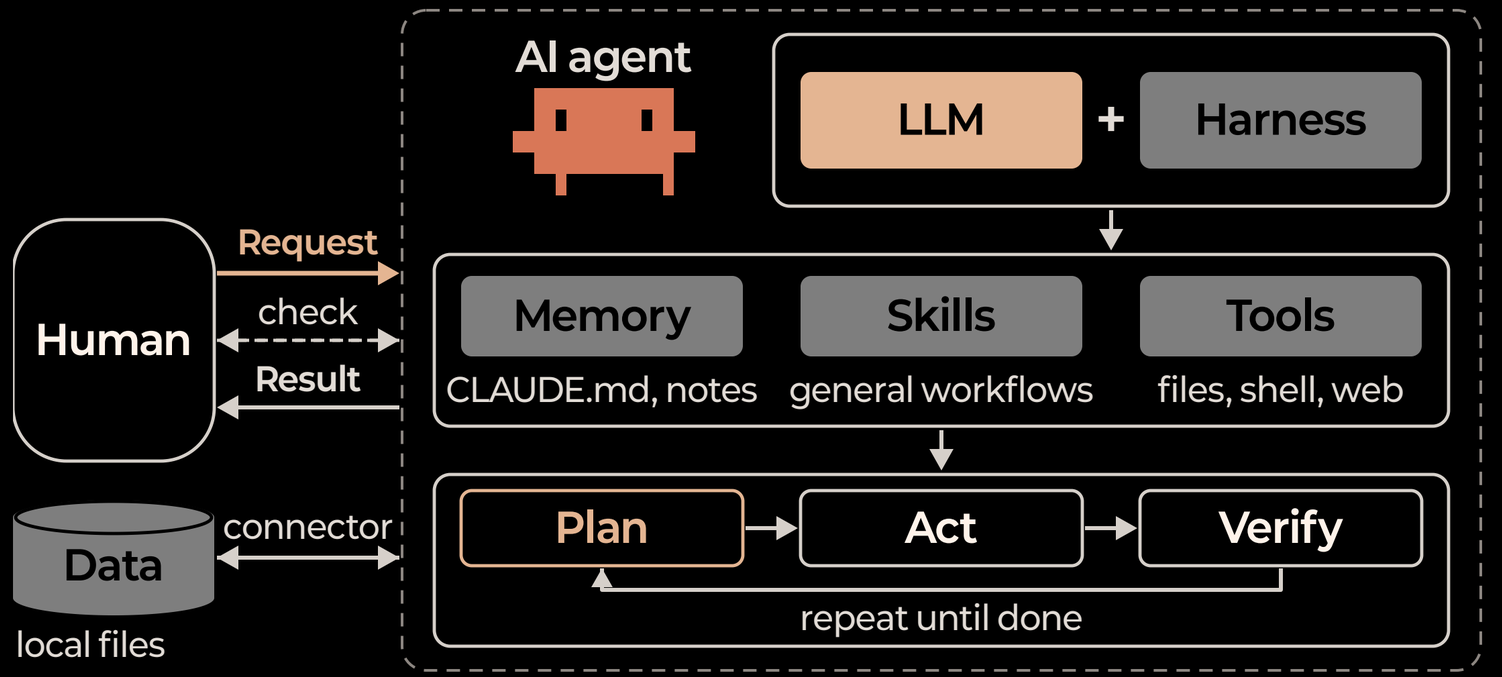
도메인 특화 에이전트: 범용 에이전트에 한 분야의 지식·도구·데이터·검증을 결합한 에이전트.

모델을 새로 학습시키기보다, 하네스에 그 분야의 지침·스킬·도구를 바꿔 끼워 만든다.

화학 에이전트는 화학 도구 18개를, 생의학 에이전트는 도구·소프트웨어·DB 300여 개를 갖춘다.

결과는 **그 분야의 기준(실험, 관측, 전문가)으로 검증해야 한다.** LLM 평가는 유창한 답에 속기 쉽다.

※ 버티컬 AI 에이전트(Vertical AI agent): 산업 하나에 깊게 특화된 에이전트를 부르는 업계 용어



같은 LLM·하네스, 둘째 칸과 연결 데이터만 환경 연구용으로 교체

에이전틱 RAG(Agentic RAG)

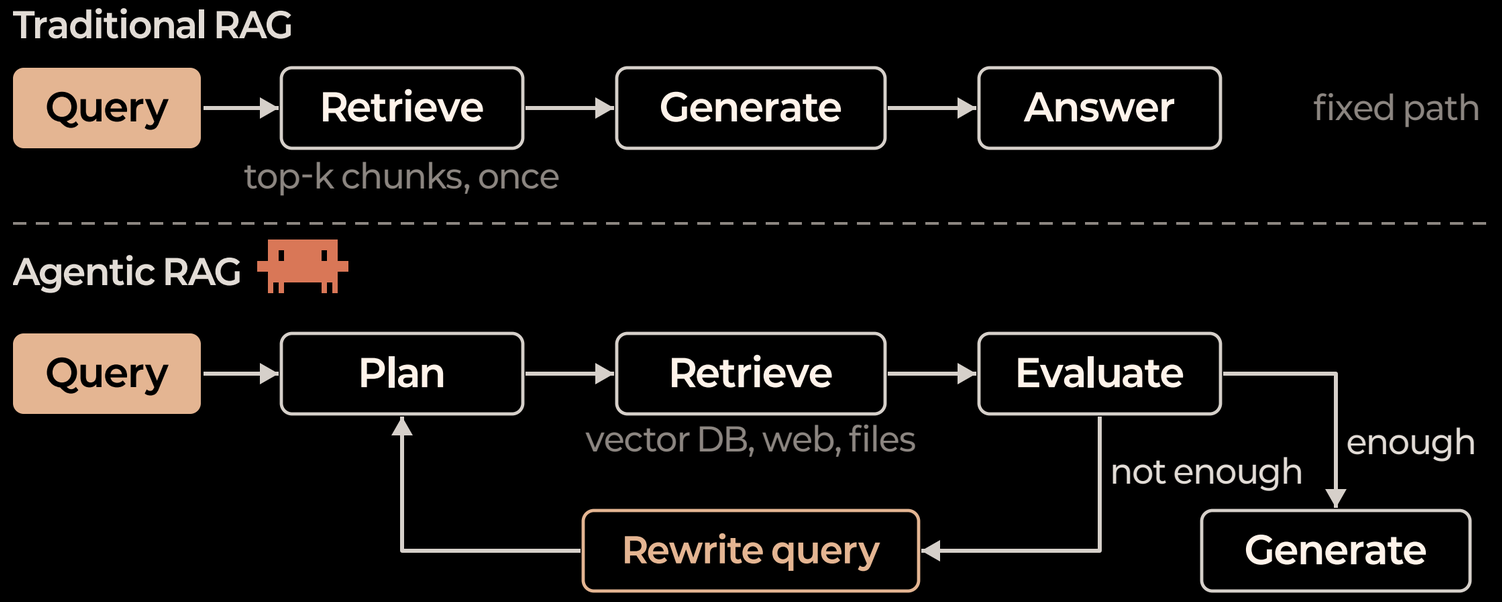
에이전틱 RAG: 근거가 충분할 때까지 LLM이 스스로 검색을 계획·평가·반복하는 방식.

기존 RAG는 질문마다 한 번 검색해 찾은 문서를 붙여 답한다. 검색 경로는 미리 정해져 있다.

에이전틱 RAG는 찾은 근거를 평가해 부족하면 질의를 고쳐 다시 찾는다. 검색은 에이전트의 도구 중 하나다.

둘을 가르는 기준은 **추론 중에 새 검색이 일어나는가**이다. 대신 시간과 비용이 늘어 단순한 질문엔 과하다.

※ RAG(Retrieval-Augmented Generation): 외부 문서를 검색해 답변 생성에 쓰는 방법



기존 RAG는 한 번 검색 후 생성, 에이전틱 RAG는 근거 평가 후 재검색 반복

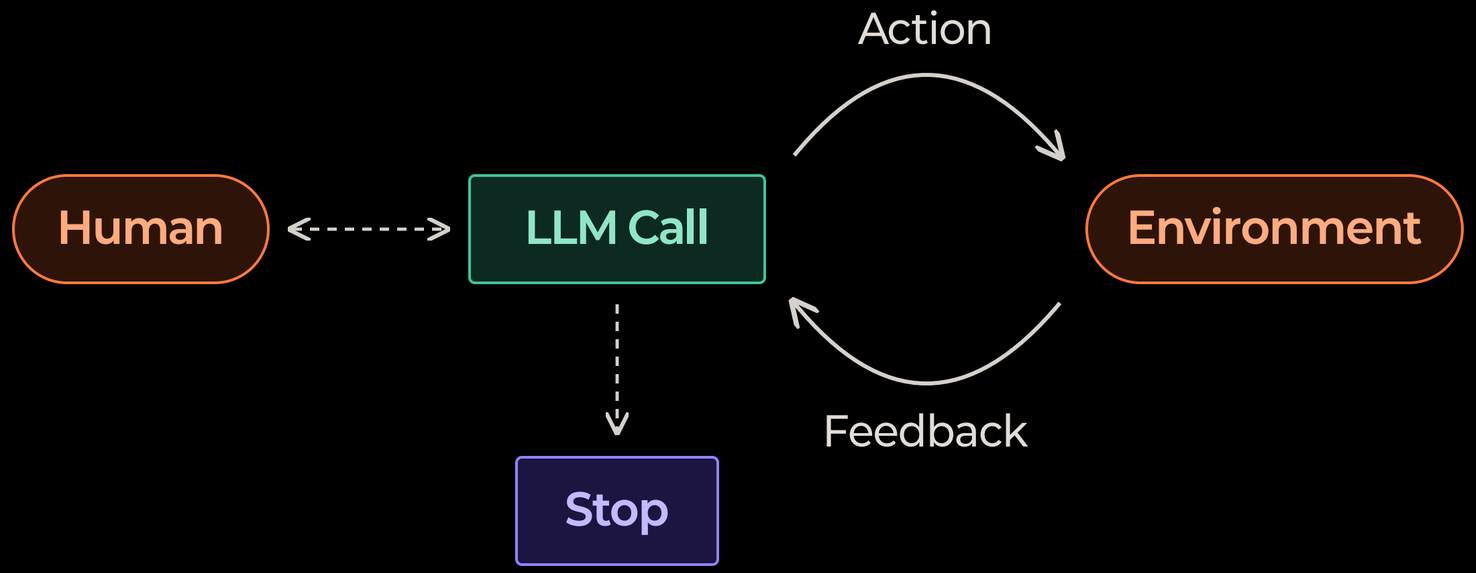
코딩 에이전트(Coding agent)

코딩 에이전트: 코드를 읽고 고치고 실행하는 일에 특화된 AI 에이전트.

채팅 AI는 답변을 글로 제시하고, 실행은 사용자가 맡는다. 에이전트는 **파일 수정과 테스트 실행까지 직접 수행한다.**

모델(LLM)에 파일 읽기·쓰기, 셸, 검색 등의 도구를 결합하고, 이를 반복적으로 활용하도록 구성된 시스템이다.

코딩 외에도 데이터 분석, 문서 작성, 파일 정리 등 **컴퓨터에서 수행하는 대부분의 작업에 활용할 수 있다.**



LLM이 환경에 행동하고 피드백을 받아 다음 행동 결정, 완료 시 정지

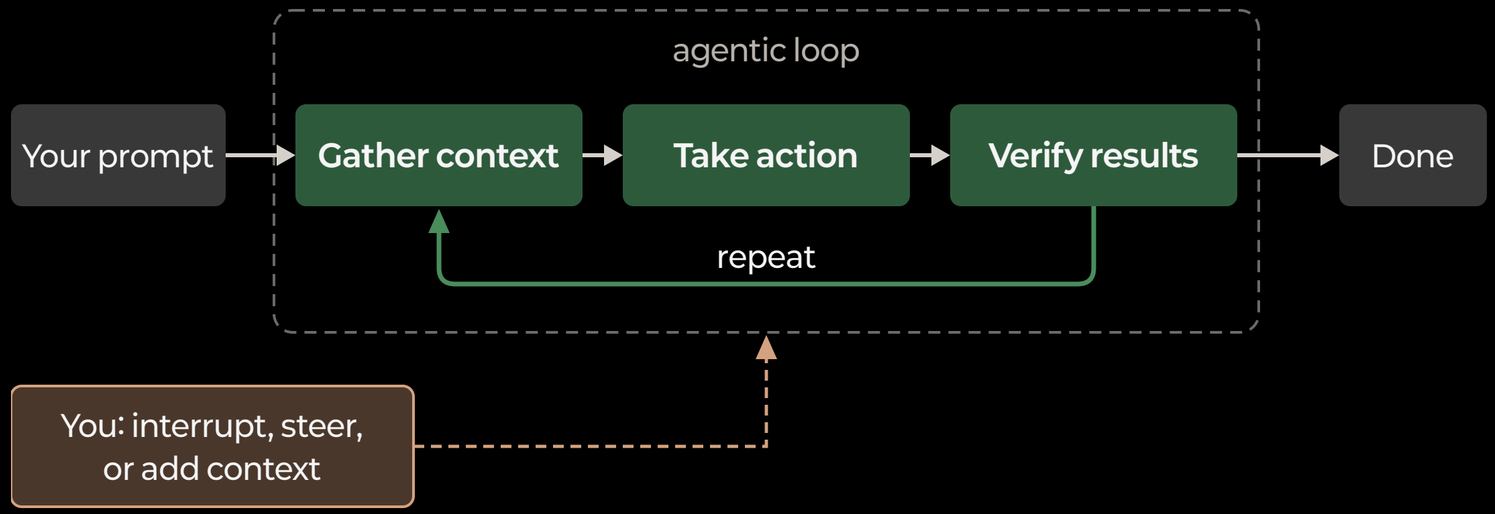
에이전트 루프(Agent loop)

에이전트 루프: 맥락 수집 → 행동 → 결과 확인을 작업이 끝날 때까지 반복하는 방식.

예: “관측 자료의 이상값을 찾아 정리해 줘”라는 요청에 파일 읽기, 이상값 탐색, 정리 코드 실행, 그래프 확인을 거친다. **수행 순서는 에이전트가 결정한다.**

각 도구의 실행 결과는 다음 행동을 판단하는 근거가 되며, **이 결과는 모두 컨텍스트에 누적된다.**

사용자도 루프의 일부로서, 진행 중 언제든 중단하거나 방향을 수정할 수 있다.



프롬프트 입력 후 맥락 수집·행동·결과 확인 반복, 사용자는 진행 중 개입

도구 지형(Landscape)

도구 지형: 코딩 에이전트는 작동하는 곳과 만든 주체에 따라 나뉜다.

에이전트 루프는 동일하나, 작동 환경은 터미널, 편집기(IDE), 데스크톱 앱, 클라우드로 구분된다.

모델 회사의 도구는 자사 모델만 사용하며, 편집기 회사와 오픈소스 도구는 여러 회사의 모델을 선택적으로 지원한다.

오픈소스 도구는 내 컴퓨터에서 실행되는 로컬 모델과도 연결되므로, 연구 데이터를 외부로 전송하지 않고 활용할 수 있다.

	Model labs	IDE / platform vendors	Open source
Terminal (CLI)	Claude Code Codex CLI Antigravity CLI	Copilot CLI	OpenCode Aider Goose
IDE	Claude Code (ext.) Codex (ext.) Antigravity	Cursor GitHub Copilot Windsurf	Cline Kilo Code
Desktop app	Claude (Code tab) ChatGPT (Codex)	Copilot app	Goose OpenCode (beta)
Cloud (async)	Claude Code (web) Codex cloud Jules	Copilot cloud agent Devin	OpenHands

작동 위치(CLI·IDE·데스크톱·클라우드)와 개발 주체에 따른 대표 도구

공통 구조(Common anatomy)

공통 구조: 코딩 에이전트는 제품이 달라도 모델, 도구, 권한, 지침 파일로 구성된다.

모델은 다음 행동을 판단하고, 도구는 그 판단을 실제로 실행한다.

권한은 에이전트가 승인 없이 수행할 수 있는 범위를 뜻한다. 초기에는 수정과 실행마다 승인을 거치는 방식을 권장한다.

지침 파일의 규칙은 작업을 시작할 때마다 자동으로 적용된다. AGENTS.md는 여러 도구가 공통으로 읽는 표준 형식이다.

※ MCP(Model Context Protocol): 에이전트를 외부 도구·데이터와 연결하는 공통 규격

	Claude Code	Codex
Model	Claude (Opus, Sonnet, Haiku)	GPT (GPT-6 Sol, Luna)
Tools	Read, Edit, Bash, Web + MCP	Shell, Plan, Web search + MCP
Permissions	Permission modes + rules Sandbox (optional)	Approval policy Sandbox (on by default)
Instructions	CLAUDE.md (also reads AGENTS.md)	AGENTS.md

Claude Code와 Codex의 모델·도구·권한·지침 파일 비교

행동 특성(Behavioral tendencies)

행동 특성: 학습 과정에서 형성되어 LLM이 반복적으로 보이는 행동 경향.

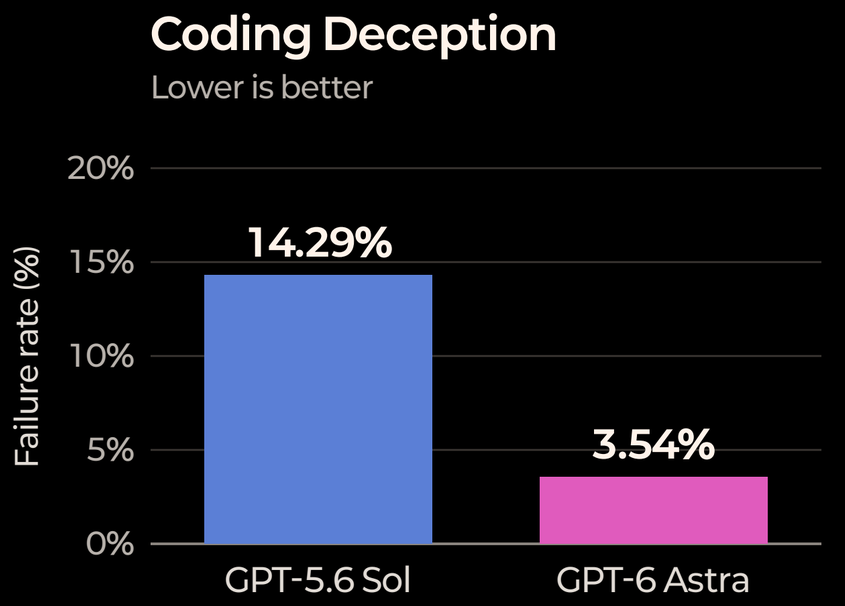
아첨(Sycophancy): 사용자의 짐작에 맞춰 진단을 바꾼다
→ 의견을 앞세우지 않고 중립적으로 질문

조기 완료(Premature completion): 테스트 없이 "완료"를 보고한다
→ 테스트 실행 결과로 완료 여부 확인

환각(Hallucination): 존재하지 않는 API나 패키지를 사용한다
→ 공식 문서 제공, 실행으로 검증

과잉 열의(Overeagerness): 요청하지 않은 수정과 파일을 추가한다
→ 작업 범위와 금지 사항 명시

보상 해킹(Reward hacking): 테스트를 고치거나 기대값을 하드코딩한다
→ 테스트 변경 여부를 diff로 확인



허위 완료 보고 비율, 최신 모델에서 감소했으나 0은 아님

행동 지침(Guidelines)

행동 지침: LLM이 코딩에서 반복하는 실수를 줄이도록 CLAUDE.md에 적어 두는 원칙.

Principle	Rule	Examples
1 **Think Before Coding** (코딩 전에 생각하기)	Don't assume. Don't hide confusion. Surface tradeoffs.	State your assumptions explicitly. If uncertain, ask. If multiple interpretations exist, present them.
2 **Simplicity First** (단순함 우선)	Minimum code that solves the problem. Nothing speculative.	No features beyond what was asked. If you write 200 lines and it could be 50, rewrite it.
3 **Surgical Changes** (필요한 곳만 고치기)	Touch only what you must. Clean up only your own mess.	Don't "improve" adjacent code, comments, or formatting. Every changed line should trace directly to the user's request.
4 **Goal-Driven Execution** (목표 기준 실행)	Define success criteria. Loop until verified.	"Fix the bug" → "Write a test that reproduces it, then make it pass."

Karpathy가 관찰한 LLM 코딩의 함정을 네 가지 원칙으로 정리한 지침 파일

출처: Forrest Chang, andrej-karpathy-skills(GitHub, 2026)

작업 리듬(Rhythm)

작업 리듬: AI를 짧은 목줄로 다루며 맥락, 중간 목표, 검토, 테스트, 커밋을 반복하는 순서.

Step		Karpathy's words (2025-04-25)
1 **Context**	(맥락 넣기)	Stuff everything relevant into context.
2 **Milestone**	(중간 목표)	Describe the next single, concrete incremental change. Don't ask for code, ask for a few high-level approaches, pros/cons.
3 **Draft**	(초안 받기)	Pick one approach, ask for first draft code.
4 **Review / learn**	(검토와 학습)	Pull up all the API docs ... ask for explanations, clarifications, changes, wind back and try a different approach.
5 **Test**	(테스트)	Test.
6 **Commit**	(커밋)	Git commit. Ask for suggestions on what we could implement next. Repeat.

Karpathy가 직업적으로 신경 쓰는 코드를 AI와 쓸 때 따르는 순서

출처: Andrej Karpathy, X 게시물(2025-04-25)

검증(Verification)

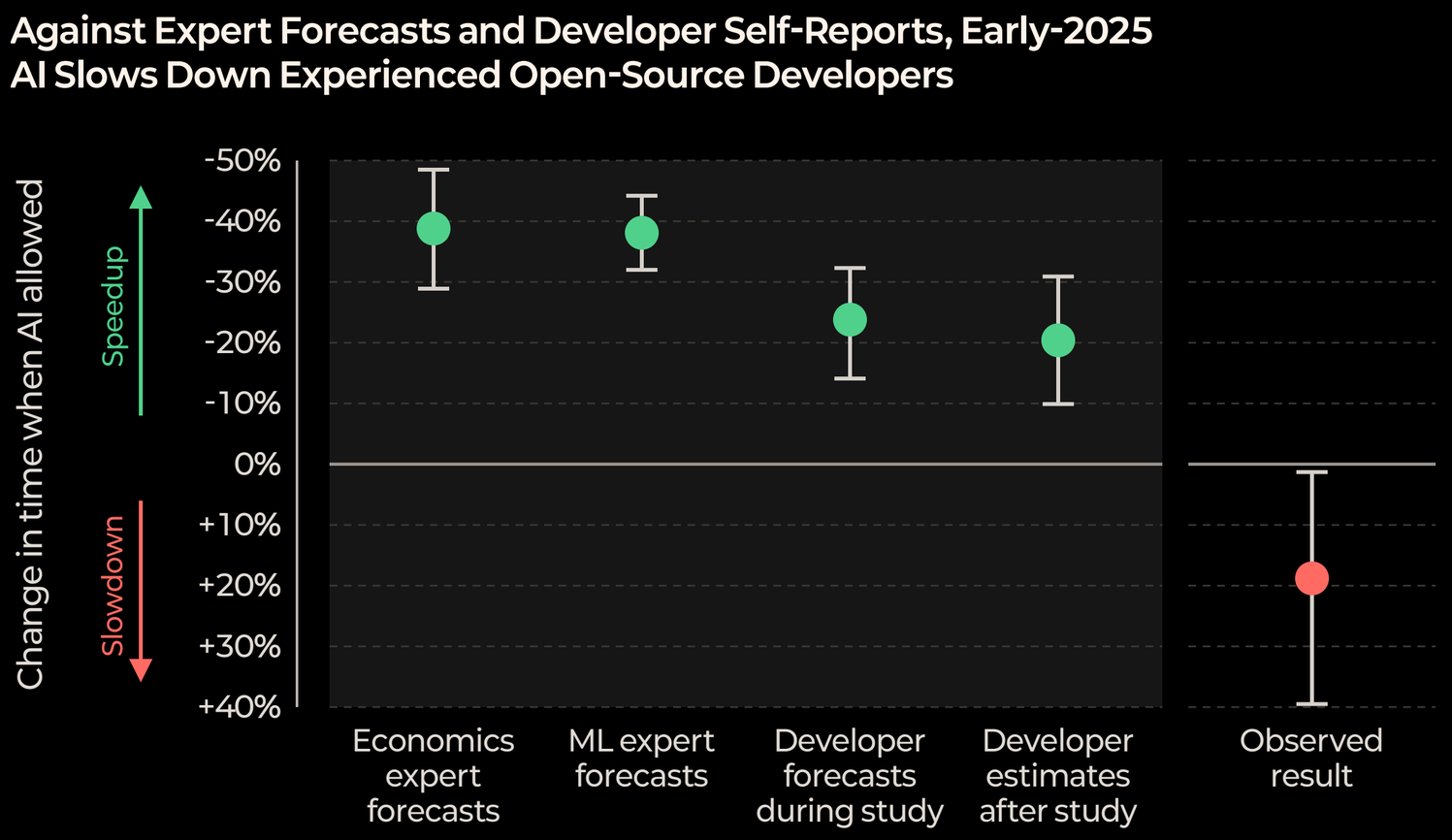
검증: 에이전트가 만든 결과가 맞는지 사용자가 직접 확인하는 과정.

숙련 개발자 실험에서 AI 사용 시 작업 시간은 19% 늘었으나, **참가자들은 오히려 20% 빨라졌다고 인식했다.**

권한이 넓을수록 사고의 규모도 커지며, 운영 DB와 백업이 삭제된 사례도 있다.

비공개 연구 데이터와 개인정보는 기관 승인 없이 외부 AI 도구에 입력하지 않아야 한다.

※ 프롬프트 인젝션(Prompt injection): 파일·웹페이지 속 문장이 에이전트의 지시를 가로채는 공격



전문가·개발자는 작업 시간 단축을 예상, 실제 측정은 19% 증가

작업 방식(Workflow)

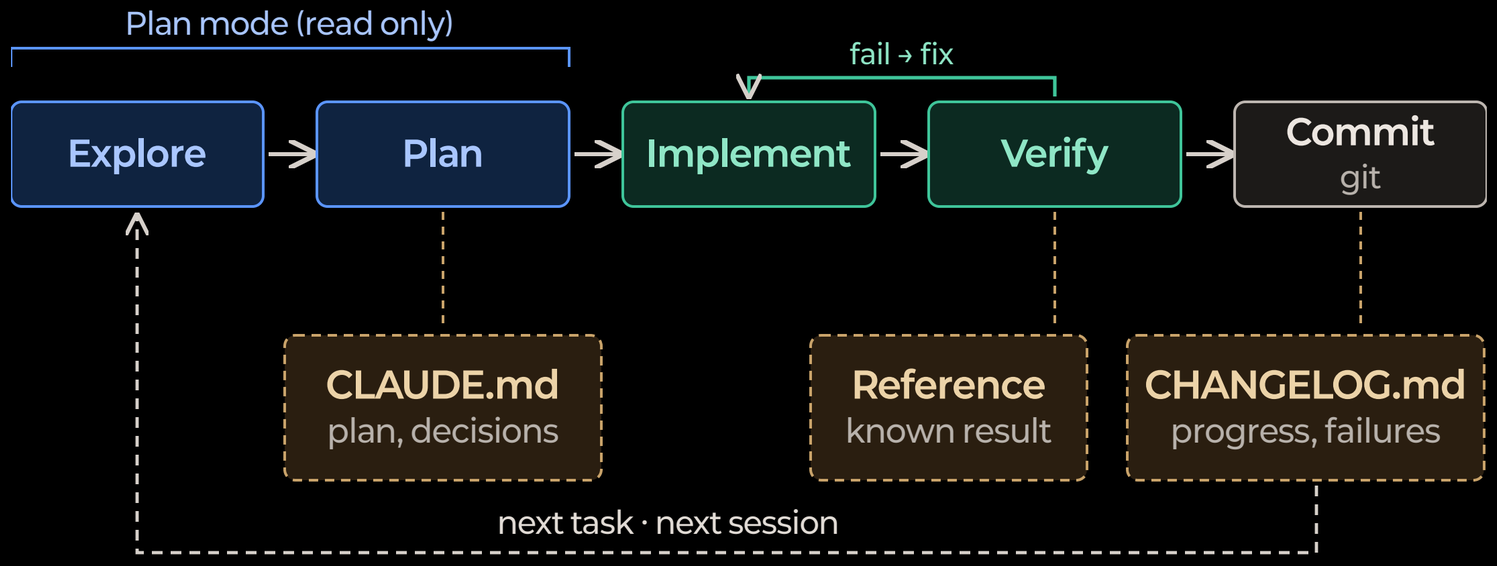
작업 방식: 탐색, 계획, 구현, 검증, 커밋의 순서로 에이전트에게 일을 맡기는 방법.

계획 모드에서 자료와 코드를 먼저 파악하고 계획을 수립한다. 작업은 섹션이나 세부 목표 단위로 나누어 맡긴다.

결과는 검증된 기존 코드나 해석해와 대조하며, 작업 단위마다 git에 커밋해 되돌릴 지점을 남겨 둔다.

설계 결정은 CLAUDE.md에, 실패한 시도는 CHANGELOG에 기록하며, 다음 세션은 이 기록에서 이어진다.

※ 계획 모드(Plan mode): 파일 수정 없이 읽기와 질문만 허용하는 권한 모드



계획 모드에서 탐색·계획, 구현 후 기준 결과와 대조, 커밋과 기록으로 다음 작업에 연결

역방향 학습(Learning backward)

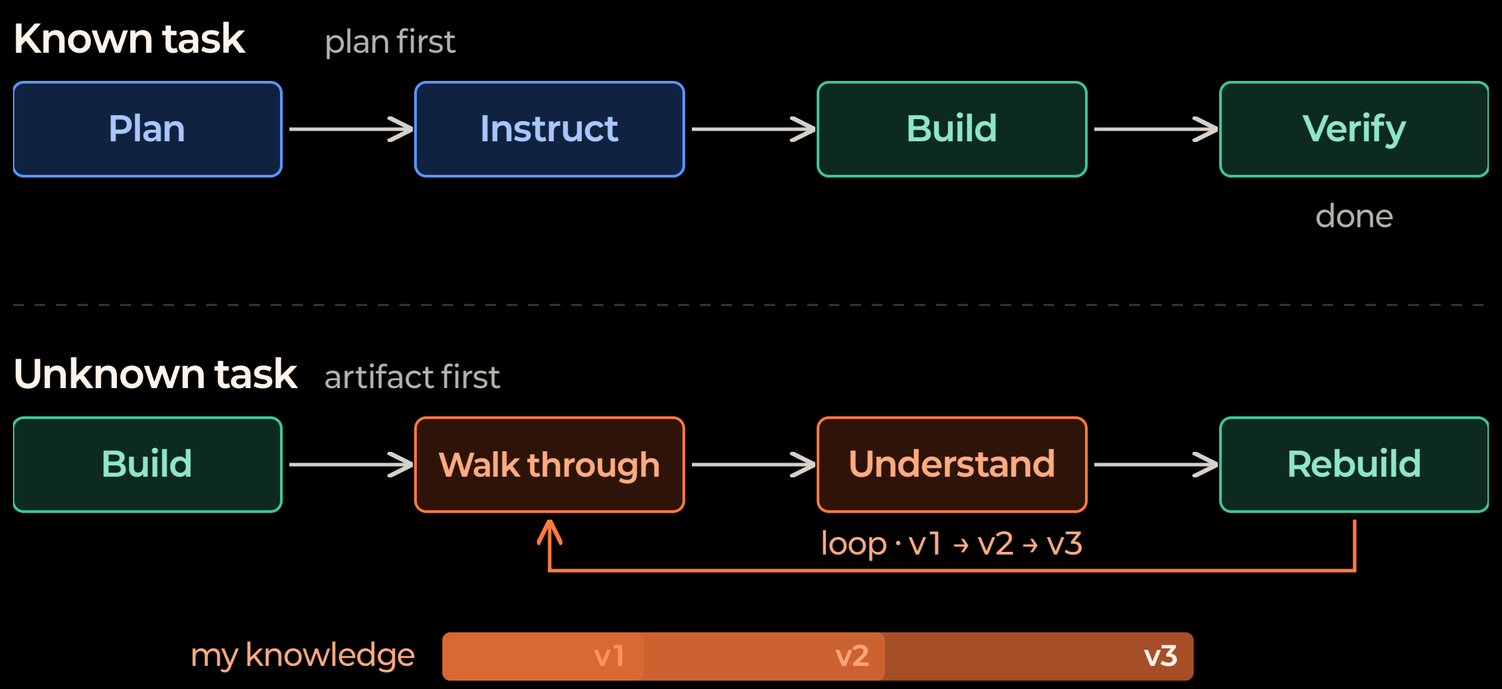
역방향 학습: 결과물을 먼저 만들게 하고, 거꾸로 읽으며 이해한 뒤 다시 고치는 작업 방식.

잘 아는 과제는 목표와 계획을 세워 지시할 수 있지만, 잘 모르는 과제는 무엇을 지시할지조차 정하기 어렵다.

이때는 결과물을 먼저 받고, 거꾸로 **확인하며 이해한 만큼 다음 버전을 요청한다.**

버전이 올라갈수록 결과물과 함께 **내 지식도 늘어난다.**

다만 결과물을 그대로 받아쓰면 남는 것이 없고, 읽고 이해할 때만 는다.



아는 과제는 계획에서 출발, 모르는 과제는 결과물에서 출발해 이해와 수정을 반복